

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа 2

Проектирование и технический дизайн API

Выполнил:

Соболев Артём

К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Описать все функциональные и нефункциональные требования к бэкенд-приложению
2. Выбрать формат реализации API: REST API или Backend For Frontend
3. Спроектировать все эндпоинты API в формате OpenAPI-схемы с учётом реализованной диаграммы БД
4. Описать тело каждого запроса и тело каждого ответа, продумать возможные ошибки, возвращаемые из API

Реализация

Акторы системы

Гость

Может:

1. Зарегистрироваться
2. Войти в систему
3. Восстановить пароль
4. Просматривать список тренировок
5. Просматривать детали тренировки
6. Просматривать блог о здоровье и питании

Авторизованный пользователь (клиент)

Может:

1. Просматривать и редактировать свой профиль
2. Изменять пароль, выходить из системы, управлять сессиями
3. Просматривать все доступные тренировки, планы тренировок
4. Просматривать пройденные пользователем тренировки
5. Добавлять тренировки в избранное
6. Оценивать тренировки
7. Проходить планы тренировок
8. Отслеживать собственный прогресс

9. Просматривать и комментировать статьи

Тренер

Может:

1. Добавлять, редактировать и удалять тренировки,
2. Добавлять, редактировать и удалять планы тренировок
3. Добавлять, редактировать и удалять упражнения
4. Добавлять, редактировать и удалять статьи о здоровье и питании

Администратор

Может:

1. Назначать роль «тренер» пользователям
2. Удалять комментарии пользователей
3. Просматривать информацию о пользователях
4. Блокировать пользователей

1. Функциональные и нефункциональные требования

1.1 Функциональные требования

1.1.1 Регистрация, аутентификация и управление учётной записью

Система должна обеспечивать:

1. Регистрацию пользователя
2. Аутентификацию пользователя
3. Возможность обновления токенов
4. Выход из текущей сессии
5. Выход со всех устройств
6. Запрос на подтверждение адреса электронной почты
7. Подтверждение адреса электронной почты
8. Запрос на сброс пароля через отправку письма на адрес электронной почты

9. Подтверждение сброса пароля
10. Просмотр профиля текущего пользователя
11. Смену пароля авторизованным пользователем
12. Получение списка активных пользовательских сессий
13. Отзыв одной выбранной сессии
14. Отзыв всех сессий, кроме текущей

1.1.2 Управление тренировками

Система должна обеспечивать:

1. Просмотр списка тренировок с пагинацией
2. Фильтрацию тренировок по уровню, типу, тегу, и поисковой строке
3. Сортировку списка тренировок
4. Просмотр детальной информации о тренировке
5. Создание тренировки тренером или администратором
6. Редактирование тренировки тренером или администратором
7. Удаление тренировки тренером или администратором
8. Привязку к тренировке упражнений, видео, типов и тегов
9. Получение рейтинга тренировки
10. Выставление оценки тренировке авторизованным пользователем
11. Изменение своей оценки тренировки
12. Удаление своей оценки тренировки

1.1.3 Управление избранным

Система должна обеспечивать:

1. Добавление тренировки в избранное
2. Удаление тренировки из избранного
3. Просмотр списка избранных тренировок пользователя с пагинацией

1.1.4 Управление упражнениями

Система должна обеспечивать:

1. Просмотр списка упражнений с пагинацией
2. Фильтрацию упражнений по части тела, уровню и поисковой строке
3. Просмотр детальной информации об упражнении
4. Создание упражнения тренером или администратором
5. Редактирование упражнения тренером или администратором
6. Удаление упражнения тренером или администратором
7. Привязку видео к упражнению

1.1.5 Управление планами тренировок

Система должна обеспечивать:

1. Просмотр списка планов тренировок с пагинацией
2. Фильтрацию планов тренировок по уровню и поисковой строке
3. Просмотр детальной информации о плане тренировок
4. Создание плана тренировок тренером или администратором
5. Редактирование плана тренировок тренером или администратором
6. Удаление плана тренировок тренером или администратором
7. Назначение плана тренировок пользователю
8. Просмотр списка назначенных пользователю планов тренировок
9. Просмотр конкретного назначенного плана тренировок
10. Обновление статуса прохождения плана тренировок
11. Хранение текущего дня и даты завершения плана тренировок

1.1.6 Учёт завершённых тренировок и прогресса

Система должна обеспечивать:

1. Сохранение факта завершения тренировки пользователем
2. Сохранение даты прохождения, длительности и количества сожжённых калорий

3. Просмотр истории завершённых тренировок пользователя
4. Расчёт и отображение пользовательского прогресса
5. Отображение общего количества тренировок, общего времени тренировок, общего количества калорий
6. Отображение текущей и максимальной серии тренировок
7. Отображение даты последней тренировки

1.1.7 Управление статьями и комментариями

Система должна обеспечивать:

1. Просмотр списка статей с пагинацией
2. Фильтрацию статей по категории и поисковой строке
3. Просмотр детальной информации о статье
4. Создание статьи тренером или администратором
5. Редактирование статьи тренером или администратором
6. Удаление статьи тренером или администратором
7. Привязку категорий и видео к статье
8. Просмотр списка комментариев к статье
9. Добавление комментария авторизованным пользователем
10. Редактирование своего комментария
11. Удаление своего комментария
12. Удаление комментария администратором

1.1.8 Управление справочными данными

Система должна обеспечивать:

1. Просмотр списка типов тренировок
2. Создание, редактирование и удаление типов тренировок тренером или администратором
3. Просмотр списка тегов тренировок

4. Создание, редактирование и удаление тегов тренером или администратором
5. Просмотр списка категорий статей
6. Создание, редактирование и удаление категорий статей тренером или администратором

1.1.9 Управление видео

Система должна обеспечивать:

1. Просмотр списка видео с пагинацией
2. Просмотр детальной информации о видео
3. Создание видео тренером или администратором
4. Редактирование видео тренером или администратором
5. Удаление видео тренером или администратором

1.1.10 Администрирование пользователей

Система должна обеспечивать:

1. Просмотр списка пользователей администратором
2. Фильтрацию пользователей по роли и поисковой строке
3. Просмотр детальной информации о пользователе
4. Изменение роли пользователя администратором
5. Блокировку и разблокировку пользователя администратором
6. Ограничение доступа заблокированных пользователей к защищённым ресурсам

1.2 Нефункциональные требования

1.2.1 Требования к безопасности

1. Доступ к защищённым эндпоинтам должен предоставляться только авторизованным пользователям с учётом ролей

2. Пароли пользователей должны храниться только в виде криптографического хэша
3. Токены подтверждения почты, токены сброса пароля и refresh token должны храниться в виде хэша
4. Токены должны иметь ограниченный срок действия
5. Должна поддерживаться возможность отзыва refresh token
6. Заблокированные или неактивные пользователи не должны иметь доступ к защищённым ресурсам
7. Система не должна раскрывать чувствительные данные в ответах API, логах и сообщениях об ошибках

1.2.2 Требования к надёжности и целостности данных

1. Система должна обеспечивать валидацию входных данных
2. API должно возвращать корректные HTTP-статусы в зависимости от результата выполнения запроса
3. Ошибки должны возвращаться в едином формате
4. Система должна обеспечивать оптимистическую блокировку для сущностей, которые могут обновляться несколькими пользователями одновременно
5. После отзыва, истечения срока действия или использования токенов повторное использование таких токенов должно быть невозможно

1.2.3 Требования к мониторингу и сопровождению

1. Система должна обеспечивать логирование основных событий
2. Логи не должны содержать конфиденциальные данные
3. Система должна поддерживать централизованную обработку ошибок

2. Формат реализации

Для данного проекта целесообразно использовать REST API, так как:

1. В проекте достаточно единого backend-сервиса для работы всех клиентов
2. Отсутствуют клиенты с существенно отличающимися требованиями к данным

При проектировании API были выделены основные функциональные модули системы:

- Аутентификация и авторизация
- Профиль пользователя
- Пользовательские сессии
- Тренировки
- Упражнения
- Планы тренировок
- Избранные тренировки
- Завершённые тренировки
- Прогресс пользователя
- Статьи
- Комментарии
- Справочные данные
- Видео
- Административное управление пользователями

3. Проектирование эндпоинтов API

Для каждого модуля были определены соответствующие эндпоинты. Например, для модуля аутентификации были предусмотрены маршруты регистрации, входа, обновления access-токена, выхода из текущей сессии и выхода со всех устройств.

Для модуля тренировок были спроектированы маршруты получения списка тренировок, получения конкретной тренировки, создания,

редактирования и удаления тренировки, а также работы с рейтингами и избранным.

Для модуля статей были предусмотрены маршруты получения списка статей, получения статьи по идентификатору, создания, изменения и удаления статьи, а также маршруты для работы с комментариями.

Для получения коллекций ресурсов предусмотрены query-параметры пагинации, сортировки и фильтрации.

Для получения конкретных сущностей используются path-параметры, например: `trainingId`, `exerciseId`, `articleId`, `commentId`, `userId`.

Для защищённых эндпоинтов была предусмотрена схема безопасности `bearerAuth`, основанная на использовании JWT access-токена.

Для обновления access-токена был спроектирован отдельный endpoint `/auth/refresh`, где refresh-токен передаётся через `HttpOnly` cookie.

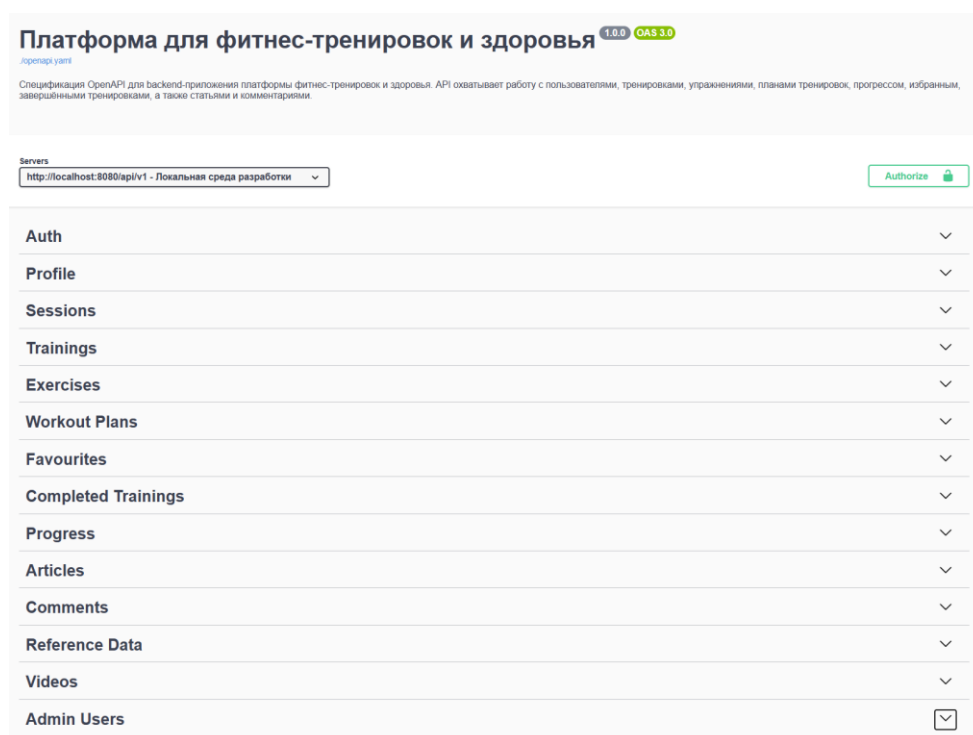


Рисунок 1 – общая структура OpenAPI-спецификации backend-приложения

Auth			^
POST	/auth/register	Зарегистрировать нового пользователя	☑
POST	/auth/login	Войти в систему	▼
POST	/auth/refresh	Обновить access-токен	▼
POST	/auth/logout	Выйти из текущей сессии	🔒 ▼
POST	/auth/logout-all	Выйти со всех устройств	🔒 ▼
POST	/auth/email-verification/request	Отправить запрос верификации почтового адреса	▼
POST	/auth/email-verification/confirm	Подтвердить верификацию почтового адреса	▼
POST	/auth/password-reset/request	Отправить запрос на восстановление пароля	▼
POST	/auth/password-reset/confirm	Подтвердить сброс пароля по токenu	▼

Рисунок 2 – группа эндпоинтов аутентификации backend-приложения

4. Описание тел запросов, тел ответов и возможных ошибок API

Для каждого эндпоинта в OpenAPI-спецификации были описаны:

- Входные параметры запроса
- Тело запроса
- Тело успешного ответа
- HTTP-статусы
- возможные ошибки

Например, для регистрации пользователя используется схема RegisterRequest, которая содержит поля username, email, password, confirmPassword. В ответе при успешной регистрации возвращается схема AuthResponse, содержащая access-токен, тип токена и краткую информацию о пользователе. Дополнительно сервер возвращает refresh-токен в заголовке Set-Cookie.

Для входа в систему используется схема LoginRequest, содержащая поля login и password. При успешной авторизации возвращается AuthResponse.

Для создания тренировки используется схема CreateTrainingRequest, включающая название тренировки, описание, уровень сложности, предполагаемую длительность, связанные типы, теги, видео и список

упражнений. При успешном выполнении возвращается объект `TrainingDetailsResponse`, содержащий полную информацию о тренировке.

Для получения списков ресурсов использовались специальные схемы страничного ответа, например `TrainingPageResponse`, содержащая информацию о номере страницы, размере страницы, общее количество элементов и страниц.

Для обработки ошибок была спроектирована единая структура ответа. Для большинства ошибок используется схема `ErrorResponse`, включающая следующие поля:

- `timestamp` – дата и время ошибки
- `status` – HTTP-статус
- `code` – внутренний код ошибки
- `message` – текст ошибки
- `path` – путь запроса
- `traceId` – идентификатор трассировки

Для ошибок валидации используется схема `ErrorValidationResponse`, которая дополнительно содержит массив `details`. Каждый элемент массива описывает конкретную ошибку поля, например имя поля и текст сообщения о нарушении ограничения.

Обрабатываются следующие типы ошибок:

- 400 Bad Request – некорректный запрос или ошибка валидации
- 401 Unauthorized – пользователь не аутентифицирован
- 403 Forbidden – недостаточно прав доступа
- 404 Not Found – ресурс не найден
- 409 Conflict – конфликт состояния ресурса

Таким образом, в спецификации были детально описаны не только успешные сценарии работы API, но и возможные ошибочные ситуации, что делает API понятным для дальнейшей реализации, тестирования и использования клиентскими приложениями.

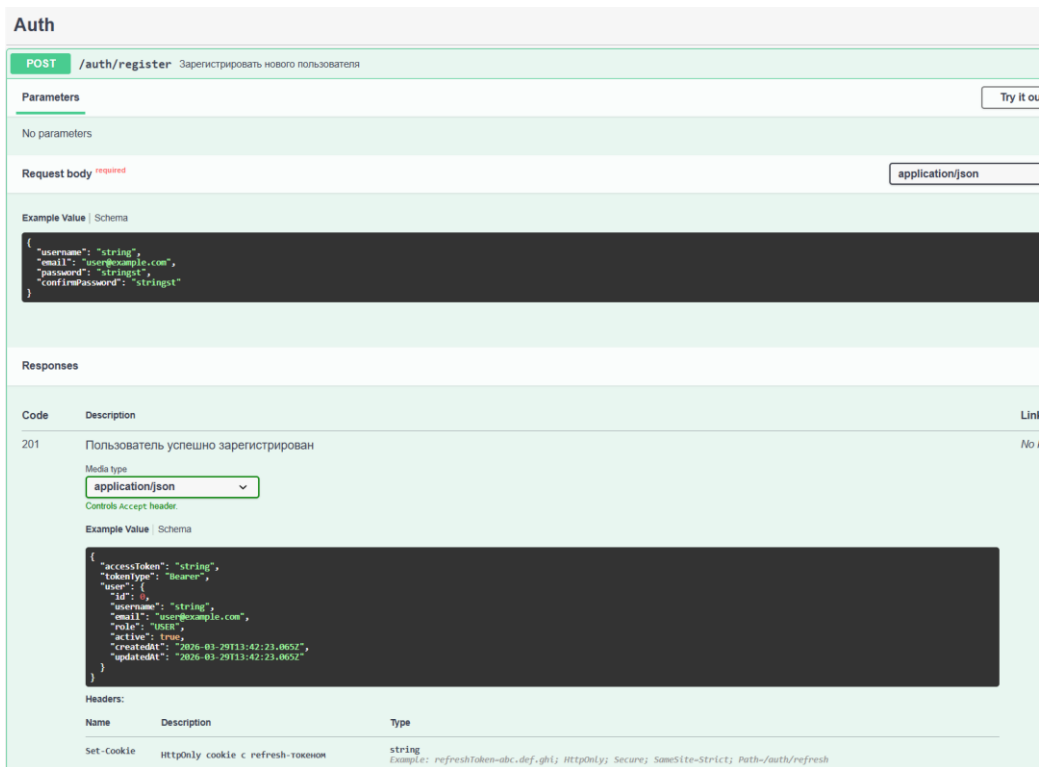


Рисунок 3 – описание тела запроса и успешного ответа эндпоинта



Рисунок 4 – описание тел ошибок

Вывод

Были определены акторы системы, сформулированы функциональные и нефункциональные требования к backend-приложению платформы фитнес-тренировок и здоровья, а также обоснован выбор формата реализации API. Для системы был выбран REST API, так как он соответствует структуре

предметной области. Были спроектированы эндпоинты API в формате OpenAPI-схемы. Для каждого эндпоинта были описаны входные параметры, тела запросов и ответов, а также возможные ошибки.